

Understanding Your LangGraph ReAct Agent

A plain-language, line-by-line walkthrough of `react.py`, `nodes.py`, and `main.py`

This document explains, in the simplest words possible, what every line of your three files is doing and **why** it's there. No fancy textbook definitions — just "this line exists because the program needs to do X."

What did you actually build? A small AI agent. You ask it a question in plain English. If the question needs a calculation or a web search, the AI figures that out on its own, uses a "tool" to get the answer, looks at the result, and decides what to do next — repeating this loop until it has a final answer for you. This pattern is called **ReAct** (short for Reason + Act): the AI reasons about what to do, acts (uses a tool), looks at the result, reasons again, and so on.

The three files work together like this:

- **`react.py`** — sets up the AI "brain" (the LLM) and gives it tools it's allowed to use.
- **`nodes.py`** — defines the two "steps" the agent can take: think, or act.
- **`main.py`** — wires those steps together into a loop (a flowchart) and actually runs it.

1. react.py — Setting up the AI's brain and tools

This file does one job: it creates the AI model (the "brain") and tells it what tools it's allowed to use. Nothing runs yet in this file — it just prepares things for the other files to use.

Line by line

```
from dotenv import load_dotenv
```

This imports a tool that reads a file called **.env** in your project folder. That file usually holds secret values like API keys (for example, your Groq API key or Tavily API key). You don't want to type API keys directly into your code — especially if you ever share the code or push it to GitHub — so instead you keep them in a hidden **.env** file and load them at runtime.

```
from langchain_core.tools import tool
```

This imports a decorator called **@tool**. A "decorator" is just a label you put above a function to give it extra powers. Here, **@tool** takes a normal Python function and turns it into something the AI model can understand and call by itself. Without this, the AI has no way of knowing your **triple()** function even exists.

```
from langchain_groq import ChatGroq
```

This imports the connector for **Groq**, which is the company hosting the actual AI model you're using (in your case, an OpenAI open-weight model called gpt-oss-120b, running on Groq's fast servers). **ChatGroq** is the object you use to talk to that model — send it messages and get replies back.

```
from langchain_tavily import TavilySearch
```

This imports a ready-made tool for searching the web, powered by a service called **Tavily**. You didn't have to write this tool yourself — someone already built it and packaged it so your agent can just plug it in and use it.

```
load_dotenv()
```

This actually runs the function you imported earlier. It reads your **.env** file and loads the keys/values inside it into your program's environment, so that when **ChatGroq(...)** or **TavilySearch(...)** run a moment later, they can automatically find the API keys they need without you passing them in manually.

The triple() tool

```
@tool
def triple(num:float) -> float:
    """
    param num: A number to be tripled.
    return: The tripled value of the input number.
    """
    return float(num) * 3
```

This is a completely normal Python function — it takes a number and multiplies it by 3. The only special thing is the **@tool** line above it. That line tells LangChain: "expose this function to the AI model, using its name, its input (num), and its docstring (the text in triple quotes) as the description." The AI actually **reads**

that docstring to decide when it should use this tool. That's why the docstring matters — it's not just a comment for humans, it's instructions for the AI.

Putting the tools together

```
tools = [TavilySearch(max_results=1), triple]
```

This creates a plain Python list containing the two tools your agent is allowed to use: a web search tool (limited to returning 1 result, to keep things fast/cheap) and your custom triple function. This list will be handed to the AI model next.

```
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0).bind_tools(tools)
```

This line does two things in one go:

1. `ChatGroq(model="openai/gpt-oss-120b", temperature=0)` — creates the AI model object, pointing at the specific model you want, with `temperature=0` meaning "be as consistent and non-random as possible" (good for tasks like this where you want predictable behavior, not creative writing).
2. `.bind_tools(tools)` — attaches your tools list to that model. This is what actually gives the AI the *option* to call `triple` or search the web. Without `bind_tools`, the model would just be a normal chatbot with no ability to use tools at all.

Think of it like this: `llm` is now a "model that knows how to ask for tools when it needs them." It doesn't run the tools itself — it just says, in its reply, "please run the triple tool with num=5" — and something else (`nodes.py`) actually executes that request.

2. nodes.py — Defining the two steps of the loop

LangGraph works by building a small flowchart made of "nodes" (steps) and "edges" (arrows connecting the steps). This file defines what happens inside two of those steps: **thinking** and **acting**.

Line by line

```
from dotenv import load_dotenv
```

Same purpose as before — load API keys from your **.env** file, in case this file is run on its own.

```
from langgraph.graph import MessagesState
```

This imports a ready-made "shape" for storing conversation data. MessagesState is basically a dictionary that always has a key called messages, holding the list of everything said so far — your questions, the AI's replies, and any tool results. LangGraph passes this state from node to node so every step can see the full conversation history.

```
from langgraph.prebuilt import ToolNode
```

This imports a ready-built "node" (step) whose entire job is: look at the last AI message, see which tool(s) it asked for, actually run those tool(s), and add the results back into the conversation. You don't have to write this logic yourself — LangGraph already built it, and you're reusing it.

```
from react import llm, tools
```

This pulls in the llm (the AI model with tools attached) and the tools list you built in **react.py**. This is exactly why react.py existed on its own — so both nodes.py and main.py can import and reuse the same setup without repeating code.

```
load_dotenv()
```

Loads the **.env** file again for this file specifically.

```
SYSTEM_MESSAGE = """
You are a helpful assistant that can answer questions and perform calculations.
"""
```

This is a plain text instruction that gets sent to the AI at the very start of every conversation, before your actual question. It tells the model what role to play. This is called the "system message" or "system prompt" — it sets the tone/behavior for everything that follows.

The "thinking" step

```
def run_agent_reasoning(state: MessagesState) -> MessagesState:
    """
    Run the agent reasoning node.
    """
    response = llm.invoke([{"role": "system", "content": SYSTEM_MESSAGE}, *state["messages"]])
    return {"messages": [response]}
```

This function is the "reasoning" step of the loop — the moment where the AI looks at the conversation so far and decides what to say or do next.

- `state: MessagesState` — the function receives the current conversation state (all messages so far).
- `llm.invoke([...])` — this actually sends a list of messages to the AI model and waits for a reply. The list is built by putting your `SYSTEM_MESSAGE` first, then unpacking (*) all the existing messages in `state["messages"]` after it.
- `*state["messages"]` — the `*` just means "take every item out of this list and place them here individually," rather than nesting the whole list inside one slot.
- `response` — whatever the AI replies with. This could be a plain text answer, OR it could be a request to use a tool (e.g. "call triple with num=5").
- `return {"messages": [response]}` — hands the new AI reply back to LangGraph so it can be added onto the conversation history.

Think of it like this: this function is basically: "show the AI everything said so far, and let it either answer, or ask to use a tool."

The "acting" step

```
tool_node = ToolNode(tools)
```

This is much shorter because it's using the ready-made `ToolNode` you imported earlier. You're simply telling it "here are the tools you're allowed to run" (the same `tools` list from `react.py`). When this node runs, it checks the AI's last message for a tool request, executes the matching Python function (like your `triple` function, or the web search), and feeds the result back into the conversation as a new message.

3. main.py — Wiring the steps into a loop and running it

This file takes the two steps you defined in nodes.py and connects them into an actual flowchart (a "graph"), then runs it with a real question.

Line by line

```
from dotenv import load_dotenv
```

Same as before — loads your API keys.

```
from langchain_core.messages import HumanMessage
```

Imports a class used to wrap a plain string as "a message that came from the human user," so the AI model can tell the difference between something you said versus something it said (AI messages), versus a tool result (tool messages).

```
from langgraph.graph import MessagesState, StateGraph, END
```

- MessagesState — same conversation-state shape used in nodes.py.
- StateGraph — the actual class you use to build your flowchart: you add nodes (steps) and edges (arrows) to it.
- END — a special built-in marker meaning "the flowchart is finished, stop here."

```
from nodes import run_agent_reasoning, tool_node
```

Imports the two step-functions you built in nodes.py, so you can plug them into the flowchart here.

```
load_dotenv()
```

Loads your **.env** file one more time, for this file.

```
AGENT_REASON = "agent_reason"  
ACT = "act"  
LAST = -1
```

These are just plain text/number labels stored in variables, so that instead of typing the string "agent_reason" everywhere (and risking a typo), you type AGENT_REASON. If you misspell a variable name, Python errors out immediately; if you misspell a raw string, it might silently fail. LAST = -1 is just a readable way of saying "give me the last item in a list" (Python lets you index from the end using negative numbers).

The decision function

```
def should_continue(state: MessagesState) -> str:  
    if not state["messages"][-1].tool_calls:  
        return END  
    return ACT
```

This function is the "traffic light" of your flowchart. After the AI thinks (the agent_reason step), LangGraph calls this function to decide what happens next.

- `state["messages"][-1]` — grabs the most recent message (the AI's latest reply).
- `.tool_calls` — every AI reply has this attribute; it's an empty list if the AI just answered normally in plain text, or a non-empty list if the AI is requesting a tool (like triple or web search).
- if not `state["messages"][-1].tool_calls`: — "if there are NO tool requests..."
- return `END` — "...then the AI is done, stop the flowchart and give the final answer."
- return `ACT` — otherwise, the AI wants to use a tool, so send control to the act step.

Building the flowchart (graph)

```
flow = StateGraph(MessagesState)
```

Creates a new, empty flowchart, and tells it that every step will be sharing the same `MessagesState` shape (the conversation history) as it passes from node to node.

```
flow.add_node(AGENT_REASON, run_agent_reasoning)
flow.set_entry_point(AGENT_REASON)
flow.add_node(ACT, tool_node)
```

- `add_node(AGENT_REASON, run_agent_reasoning)` — registers a step named "agent_reason" that, when reached, runs your `run_agent_reasoning` function (the thinking step).
- `set_entry_point(AGENT_REASON)` — tells the flowchart "always start here" — this is the `__start__` arrow you see pointing into `agent_reason` in your diagram.
- `add_node(ACT, tool_node)` — registers a second step named "act" that runs `tool_node` (the acting step) when reached.

```
flow.add_conditional_edges(AGENT_REASON, should_continue, {
    END: END,
    ACT: ACT
})
```

This is the most important line for the branching arrows you see in your diagram (the two dotted lines coming out of `agent_reason`). It says: "after the `agent_reason` step finishes, run the `should_continue` function to decide where to go next." The dictionary `{END: END, ACT: ACT}` is just a lookup table mapping what `should_continue` returned to which node to actually jump to. Here it's a 1-to-1 mapping (the value returned is the node name), but `LangGraph` requires you to spell this mapping out explicitly.

```
flow.add_edge(ACT, AGENT_REASON)
```

This adds a plain, unconditional arrow: "after the act step finishes running a tool, always go back to `agent_reason`." This is what creates the **loop** in your diagram — the solid arrow from `act` back up to `agent_reason`. It lets the AI see the tool's result and think again (maybe it needs another tool, or maybe it's ready to answer now).

```
app = flow.compile()
```

Turns your flowchart *definition* into something you can actually run. Up until now you were just describing the shape of the graph; `compile()` locks it in and returns a runnable object, `app`.

```
app.get_graph().draw_mermaid_png(output_file_path="flow.png")
```

This is purely for visualization — it asks LangGraph to draw your compiled flowchart as an image and save it to flow.png. That's exactly the picture you shared: `__start__` → `agent_reason` → (either `__end__` or `act`) → back to `agent_reason`. This line has nothing to do with the agent actually answering questions; it just documents the flow visually.

Actually running it

```
if __name__ == "__main__":
    print("Hello ReAct LangGraph with function calling")
    res = app.invoke({"messages": [HumanMessage(content="What is the temperature in Ghaziabad? List it and then triple it")]}))
    print(res["messages"][LAST].content)
```

- `if __name__ == "__main__":` — a standard Python safety check meaning "only run the code below if this file was run directly (like `python main.py`), not if it was imported by another file."
- `app.invoke({...})` — this is the moment everything actually executes. You're handing the flowchart a starting state: one message from you (wrapped in `HumanMessage`) asking about the temperature in Ghaziabad and to triple it.
- `res` — the final state after the flowchart finishes running (after it reaches `END`). It contains the full message history: your question, the AI's tool requests, the tool results, and the final answer.
- `res["messages"][LAST].content` — grabs the very last message in that history (using the `LAST = -1` shortcut from earlier) and prints just its text content — the AI's final answer to you.

4. What actually happens, step by step, when you run main.py

Here's the real sequence of events for the example question: "What is the temperature in Ghaziabad? List it and then triple it."

- 1 You run `python main.py`. Python executes `main.py` top to bottom, which imports `nodes.py`, which imports `react.py` — so all the setup (loading keys, creating the model, building the graph) happens first.
- 2 `app.invoke(...)` starts the flowchart at `agent_reason`, with your question as the only message so far.
- 3 Inside `run_agent_reasoning`, the AI reads your question and realizes it needs live weather data — something it doesn't know on its own. So instead of replying with an answer, it replies with a **tool request**: "please call the web search tool with a query about Ghaziabad's temperature." This reply has `tool_calls` filled in.
- 4 `should_continue` checks the AI's last message, sees `tool_calls` is NOT empty, and returns `ACT` — so the flowchart moves to the `act` node.
- 5 `tool_node` (built from `ToolNode`) actually runs the web search tool with the query the AI asked for, gets back a result (the current temperature), and adds it to the conversation as a new message.
- 6 The graph's fixed edge sends control back to `agent_reason` automatically.
- 7 `run_agent_reasoning` runs again — but this time the AI can see the search result in the conversation. It now knows the temperature, and your question also asked to triple it, so this time it replies with another tool request: "call triple with num=<the temperature>."
- 8 `should_continue` again sees `tool_calls` present, sends it to `act` again.
- 9 `tool_node` runs your triple function on the temperature, adds the tripled number as a new message.
- 10 Back to `agent_reason` one more time. Now the AI has both the temperature and the tripled value in front of it, and no more information is missing — so this time it replies with a normal text answer instead of a tool request.
- 11 `should_continue` sees `tool_calls` is empty this time, and returns `END` — the flowchart stops.

12 `app.invoke(...)` returns the full conversation. `print(res["messages"][LAST].content)` prints the AI's final plain-text answer to your terminal.

Think of it like this: this whole file is just a loop: *think* → *(maybe use a tool)* → *look at the result* → *think again* — repeating until the AI has everything it needs to answer you directly. Your flow.png diagram is literally a picture of this loop.

5. Why split this into three separate files at all?

You could technically put everything in one file, but splitting it up like this is a common, sensible pattern:

- **react.py** = configuration. "What model am I using, and what tools does it have?" You'd change this if you switched from Groq to OpenAI, or added a new tool.
- **nodes.py** = behavior. "What does each step in the loop actually do?" You'd change this if you wanted to change the system prompt, or add logging, or change how tools get executed.
- **main.py** = wiring + running. "How do the steps connect into a flowchart, and what question do I actually ask?" You'd change this if you wanted to change the shape of the flowchart itself, or run a different question.

This separation means each file has one clear job, and you can change one part (like swapping in a different AI model in react.py) without having to touch the flowchart logic in main.py at all.